

Lab 02 — Fila Fácil

Tutorial prático: do pedido ao protótipo com um agente de IA

TEC.1053 · ADS IV · IFPI · 2026/2 · Prof. Aislan Rafael Unidade 2 — Assistentes, agentes e ferramentas ·
Aula de referência: 08/09/2026

PASSO 01

1. O que vamos construir

Imagine a recepção de uma clínica: uma pessoa chega, retira uma senha e aguarda a chamada. Vamos colocar esse fluxo para funcionar em uma página. A versão básica atende por ordem de chegada. Prioridade fica para o exercício de evolução, depois do fluxo básico verificado.

Ao final você terá um protótipo que emite N001, N002...; chama a próxima senha; mostra a fila de espera e o total de chamadas; e reinicia somente após confirmação.

O ponto de partida já traz a interface e os botões conectados. Faltam três operações em `src/fila.js`. Você fará a IA trabalhar em incrementos pequenos, observará as ações e verificará cada resultado. O objetivo é conseguir explicar o processo e o código.

Modelo gera a resposta ou solicita uma ação. **Harness**, como o OpenCode, executa as ferramentas disponíveis. No **chat**, você faz a ponte: cola contexto, aplica código e devolve resultados. Nos dois caminhos, a revisão é sua.

2. Prepare uma cópia sem perder o Lab 01

Use o terminal que já funcionou no laboratório anterior. Você precisa de Git, uv e um navegador. O projeto usa HTML, CSS e JavaScript nativos: não instale Node, npm, React, banco de dados ou pacotes para completar este tutorial. Node é apenas uma alternativa opcional para executar testes no terminal de quem já o possui.

Caminho recomendado para o primeiro ensaio: cópia nova

Abra o terminal na pasta em que guarda seus projetos, fora de outro clone, e execute:

```
git clone https://github.com/aislanifpi/tec1053.git tec1053-lab02
cd tec1053-lab02
git status --short
```

O último comando não deve listar alterações. Se a pasta já existir, escolha outro nome para o destino do clone. A cópia antiga e a solução do Lab 01 ficam preservadas.

Se quiser atualizar uma cópia existente

Rode `git status --short` e `git remote -v`. Se houver alterações ou dúvida sobre a branch, use a cópia nova acima. Não use `reset` ou comandos de descarte para “resolver”. Em uma cópia limpa da branch `main` cujo origin seja o repositório da disciplina:

```
git pull --ff-only origin main
```

Em um fork, confirme se existe um remoto upstream apontando para `aislanifpi/tec1053`. Se não existir, adicione-o com `git remote add upstream https://github.com/aislanifpi/tec1053.git`. Na branch `main` limpa, use `git pull --ff-only upstream main`. Se houver divergência, pare a atualização e use um clone novo. Não presuma que qualquer pull será sem conflito.

Checkpoint: na raiz, você deve ver `AGENTS.md` e a pasta `labs/lab-02-fila-facil`. Se a pasta do lab não aparecer, confira a atualização antes de seguir.

3. Abra a aplicação e registre a linha de base

Na raiz do repositório, execute:

```
uv run python -m http.server 8000 --bind 127.0.0.1 -d labs/lab-02-fila-facil
```

Mantenha esse terminal aberto. É um servidor de arquivos na sua própria máquina; não publica a aplicação na internet. Abra no navegador:

- Aplicação: <http://127.0.0.1:8000/>
- Testes: <http://127.0.0.1:8000/tests/>

O painel deve abrir com “—”, fila vazia e zero senhas chamadas. Os três botões ainda não executam as regras pendentes. O aviso de implementação pendente é esperado no esqueleto.

Na página de testes, a linha de base correta é:

6 aprovados · 18 falhas · 24 testes

Os seis testes verdes cobrem estado inicial e formatação já fornecidos. As falhas indicam as operações ainda não implementadas. Registre a linha e uma captura. Tela em branco ou erro de carregamento não é a linha de base esperada.

Alternativa sem servidor: como não há módulos nem chamadas de rede, você pode abrir `index.html` e `tests/index.html` diretamente pelo gerenciador de arquivos. O caminho com servidor é o recomendado para manter a turma no mesmo endereço de trabalho.

Se a porta estiver ocupada: troque 8000 por 8001 no comando e nos dois endereços. Encerre o servidor com Ctrl+C quando terminar. Abra um segundo terminal na mesma raiz para os próximos comandos; não digite prompts da IA no terminal ocupado pelo servidor.

4. Entenda os arquivos antes de pedir código

Arquivo ou pasta	Para que serve
AGENTS.md da raiz	Regras gerais da disciplina.
labs/lab-02-fila-facil/AGENTS.md	Escopo específico do Lab 02.
ESPECIFICACAO.md do lab	Contrato das operações e critérios de verificação.
src/fila.js	Único arquivo de código que você altera no fluxo básico.
index.html, src/app.js, src/estilo.css	Interface fornecida e ligação dos botões.
tests/	Contrato comum de 24 testes. Não alterar.
prompts/ e evidencias/	Registro real dos pedidos, resultados e intervenções.

Abra os dois AGENTS.md e a especificação inteira. Localize as cinco funções do objeto FilaFacil: criarEstado e formatarSenha já estão prontas; emitirSenha, chamarProxima e reiniciarFila são a tarefa. O estado tem proximaSenha, aguardando, atual e totalChamadas. As operações devem alterar o objeto recebido; só devolver um objeto novo não basta.

Checkpoint: explique por que emitir uma senha não deve mudar a senha em atendimento.

5. Escolha uma rota de IA que já esteja funcionando

Rota A — OpenCode ou outro agente com ferramentas

No segundo terminal, na raiz do repositório, execute `opencode`. Use o acesso que já funcionou no Lab 01. No OpenCode, `/connect` abre a conexão com provedores e `/models` permite escolher um modelo entre as opções disponíveis na instalação.

Ferramenta, provedor e modelo são escolhas diferentes. Uma chave autentica o acesso; a existência de uma chave não prova que há cobrança. Assinatura de chat e acesso por API também não são a mesma coisa. Verifique as condições do acesso já disponível. Não é necessário comprar serviços para este laboratório. Nunca cole uma chave no prompt, no código, no repositório ou na evidência.

Não gastem a aula tentando fazer um modelo local sustentar o laço. Se a ferramenta pedir contratação, se o acesso não funcionar ou se o modelo imprimir uma chamada de ferramenta como texto sem executá-la, passem para a rota B ou trabalhem em dupla com ambiente preparado. Registrem o motivo. Não vamos pressupor que um modelo listado esteja gratuito ou disponível para todos.

Rota B — Claude, ChatGPT ou outro chat com edição manual

Abra o chat que você já consegue usar. Anexe ou cole os dois `AGENTS.md`, a especificação e o conteúdo atual de `src/fila.js`. Acrescente a instrução abaixo antes de cada prompt:

```
Você está em um chat sem acesso ao meu computador. Os documentos e o código seguem anexados/colados. Não afirme ter lido arquivos locais, editado ou executado testes. Explique a mudança desta etapa e devolva somente a função solicitada, preservando sua assinatura. Eu aplicarei a mudança e enviarei a saída real dos testes.
```

Cole somente a função correspondente no arquivo, salve e teste. Uma resposta em chat não edita automaticamente seu projeto. Não copie cercas de Markdown para o JavaScript.

6. Primeiro pedido: inspecionar, sem alterar

Na rota A, envie este prompt. Na rota B, envie junto com os arquivos da etapa anterior.

```
Vamos trabalhar somente no Lab 02 – Fila Fácil.  
Leia AGENTS.md da raiz, labs/lab-02-fila-facil/AGENTS.md, a especificação completa  
e src/fila.js desse lab. Ainda não edite arquivos.  
Resuma o contrato, identifique as três funções pendentes e proponha três incrementos.  
Explique como verificar cada um e quais arquivos você não pode alterar.  
Se tiver ferramentas, mostre as leituras reais. Não imprima chamadas fictícias.
```

Confira se o agente realmente usou leitura de arquivos e se o plano mantém o escopo. Se ele propuser framework, servidor de aplicação, banco ou alterações no Lab 01, corrija o plano antes da implementação. A página já tem uma interface funcional como estrutura.

Registro: salve o prompt efetivamente usado em prompts/01-inspecao.md dentro do lab.

7. Incremento 1 — emitir uma senha

Implemente somente o corpo de `emitirSenha` em `labs/lab-02-fila-facil/src/fila.js`, conforme RF-02. Preserve o restante do arquivo, a interface e os testes. Não implemente as outras duas funções agora. Explique o efeito em cada campo do estado. Se Node já estiver disponível, execute `node labs/lab-02-fila-facil/tests/executar.mjs` na raiz e relate a saída real. Se não estiver, eu executarei a suíte no navegador; não instale dependências nem declare testes aprovados sem execução.

Após a alteração, salve e recarregue a aplicação. Clique três vezes em **Retirar senha**: a fila deve mostrar N001, N002, N003, a última emissão N003 e o painel atual “—”. A quantidade em espera deve ser 3 e o total de chamadas 0.

Recarregue também a página de testes. Com apenas essa função implementada corretamente, o resultado esperado é **10 aprovados · 14 falhas**. Salve o prompt e a evidência reais.

No segundo terminal, fora da conversa da IA, confira:

```
git diff -- labs/lab-02-fila-facil/src/fila.js
git status --short
```

Checkpoint: o que você observou que comprova edição real, além da resposta da IA?

8. Incremento 2 — chamar na ordem correta

Agora implemente somente `chamarProxima` em `labs/lab-02-fila-facil/src/fila.js`. Cumpra RF-03 e RF-04: atender FIFO, retirar da espera, atualizar a senha atual e contar chamadas efetivas. Fila vazia retorna null e preserva todo o estado. Preserve `emitirSenha`, a interface, `reiniciarFila` e os testes. Verifique pelo caminho disponível e explique como tratou o caso vazio. Não acrescente prioridade ainda.

Recarregue a aplicação, emita três senhas e chame uma. Deve aparecer N001 em atendimento, N002 e N003 na espera, quantidade 2 e total de chamadas 1. Chame mais duas vezes. Depois clique mais uma vez com a fila vazia: o painel deve continuar em N003, o total em 3 e a mensagem deve informar que não há senhas aguardando.

Nos testes, com emissão e chamada corretas, espere **20 aprovados · 4 falhas**. As quatro restantes dependem do reinício. Se a sua saída for diferente, registre-a.

Se algo falhar: intervenha com evidência

Pare de ampliar a solução. A verificação produziu esta falha: [cole a saída real]. Relacione-a ao requisito, explique a causa e proponha a menor correção em `src/fila.js`. Não altere os testes nem esconda o erro com `try/catch`. Verifique novamente depois.

Observe se o agente lê a falha ou repete a mesma alteração. Uma chamada de ferramenta malformada, erro de acesso e regra de negócio errada são problemas diferentes. Interrompa pelo controle de cancelamento da ferramenta se entrar em repetição.

9. Incremento 3 — reiniciar com controle humano

Implemente somente `reiniciarFila` em `labs/lab-02-fila-facil/src/fila.js`, conforme RF-05. Restaure os quatro campos no mesmo objeto recebido. A confirmação já é feita pela interface; não coloque `confirm()`, HTML ou eventos nesta função. Preserve as operações anteriores e a suíte. Execute a verificação disponível e informe o resultado real.

Recarregue, emita duas senhas e chame uma. Clique **Reiniciar demonstração** e escolha **Cancelar**: tudo deve permanecer igual. Repita e confirme: atual “—”, espera vazia, zero chamadas e comprovante oculto. A próxima emissão deve ser N001.

Agora a suíte deve mostrar **24 aprovados · 0 falhas · 24 testes**. Isso é uma meta a comprovar, não um resultado que você deve copiar para a entrega.

10. Faça a verificação final fora da conversa da IA

Troquem os papéis da dupla. Quem revisa deve executar a suíte e este roteiro visual:

Ação	Resultado esperado
Recarregar a aplicação	“—”, espera 0 e chamadas 0.
Emitir três senhas	N001, N002, N003, sem chamada automática.
Chamar uma	N001 atual; N002 e N003 aguardando; total 1.
Emitir outra	N004 entra ao fim da espera.
Chamar até esvaziar	Ordem N002, N003, N004; total 4.
Chamar vazia	Preserva N004 e total 4; mostra aviso.
Cancelar reinício	Nenhum dado muda.
Confirmar reinício	Limpa tudo; próxima emissão N001.
Usar Tab e Enter	Botões acessíveis por teclado; foco visível.
Recarregar ou abrir outra aba	Nova sessão vazia; abas não sincronizam.

Confira também no terminal:

```
git diff -- labs/lab-02-fila-facil/src/fila.js
git diff -- labs/lab-02-fila-facil/tests
git status --short
```

O diff dos testes deve estar vazio. O status deve mostrar somente o arquivo autorizado e seus registros em prompts/ e evidencias/. Confira também alterações já preparadas para commit, se houver, com `git diff --cached`. Não aceite mudanças em outro laboratório.

Quem já tem Node 18 ou superior pode executar a mesma suíte com:

```
node labs/lab-02-fila-facil/tests/executar.mjs
```

O comando `uv run pytest` do repositório continua referente aos testes Python do Lab 01. Os 53 testes do validador não substituem os 24 testes JavaScript do Fila Fácil.

11. Registre o resultado e prepare a evolução

Em evidencias/resultado.md, dentro do lab, registre:

- Dupla, data, rota, ferramenta e modelo, quando utilizado.
- Resumos inicial e final dos testes, sem inventar resultados.
- Capturas do protótipo e conclusão do roteiro visual.
- Arquivos alterados e uma decisão da implementação explicada pela dupla.
- Uma intervenção humana ou revisão que foi necessária e o motivo.
- Dúvidas e limitações restantes, inclusive se não concluiu.

Salve os prompts reais numerados em prompts/. Nunca registre chaves ou dados pessoais. Com tudo revisado, você pode criar um commit local somente do lab:

```
git add labs/lab-02-fila-facil/src/fila.js
git add labs/lab-02-fila-facil/prompts
git add labs/lab-02-fila-facil/evidencias
git diff --cached --stat
git commit -m "Conclui fluxo básico do Lab 02 – Fila Fácil"
```

Se Git pedir identidade, configure seu nome e e-mail apenas conforme a orientação da disciplina. Não é preciso fazer push para demonstrar a aplicação na aula.

Próximo desafio: classificar senhas por prioridade administrativa. Antes de editar, defina ordem dentro de cada categoria e limite de chamadas prioritárias consecutivas. O enunciado de evolução está em EXERCICIOS.md. Não misture essa mudança com o fluxo básico.

12. Problemas frequentes e saída rápida

Situação	Ação
Pasta do Lab 02 não existe	Atualize com segurança ou use um clone novo.
uv não é reconhecido	Use o ambiente do Lab 01 ou abra os HTML diretamente; não gaste a prática reinstalando tudo.
Porta 8000 ocupada	Use 8001 no servidor e no navegador.
Página não abre	Confira o terminal do servidor, a raiz e o endereço completo.
Código mudou, resultado não	Salve, recarregue aplicação e testes; confira se editou o clone que está sendo servido.
Página em branco / erro de sintaxe	Abra o console do navegador; procure cercas Markdown ou função incompleta no arquivo.
Botão diz “ainda precisa ser implementada”	A função correspondente ainda tem o marcador LAB02_PENDENTE.
Agente pede chave	Confira o provedor e o acesso disponível; chave não significa automaticamente cobrança. Use chat se necessário.
Agente só explica ou imprime JSON	Ele não executou uma ferramenta; use edição manual ou outra rota já disponível.
Testes foram alterados	Não aceite o resultado; preserve suas evidências e recupere o contrato a partir de cópia limpa com orientação.
Testes verdes, botão errado	Revise a integração e o roteiro visual. A suíte de domínio não cobre toda a interface.

Fontes e escopo

Contrato: ESPECIFICACAO.md deste laboratório. Instruções do OpenCode consultadas em 08/09/2026: <https://opencode.ai/docs/cli/> e <https://opencode.ai/docs/providers/>. Disponibilidade de modelos e autenticação podem variar. Nenhum provedor específico é obrigatório. Este tutorial não pressupõe execução bem-sucedida de um modelo local.